

Q-Flow: Bulut Tabanlı, Dinamik ve Uçtan Uca Takip Edilebilir QR Kod Yönetim Sistemi

MTH2526-B25 Bulut Mimarilerinde Test Mühendisliği Dönem Projesi Raporu

Öğrenciler: Şevval Çulcu (171423961) | Yusuf Durmuş (170423965)

Eğitmen: Büşra Ayaksız

Tarih: Haziran 2026

1. Giriş

Geleneksel statik QR kodları, basıldıktan sonra içerdikleri hedef bağlantıların (URL) değiştirilememesi ve kullanıcılar tarafından kaç kez tarandığının takip edilememesi gibi ciddi işlevsel kısıtlamalara sahiptir. Bu durum, basılı materyallerin (örneğin restoran menüleri, reklam panoları veya broşürler) güncellenmesi gerektiğinde yüksek operasyonel maliyetlere ve veri kaybına yol açmaktadır.

Bu projede geliştirilen **Q-Flow**, statik bağlantı kısıtlamalarını ortadan kaldıran dinamik bir yönlendirme katmanı sunar. Projenin ana amacı, karmaşık bir iş mantığı geliştirmekten ziyade; bulut-yerel (cloud-native) standartlarda çalışan bir mikroservisin tüm yazılım yaşam döngüsünü (SDLC) endüstriyel test otomasyon standartlarına uygun olarak tasarlamak ve test etmektir. Dönem projesi kapsamında birim (unit), entegrasyon (integration), uçtan uca (E2E), API ve performans test otomasyon zincirlerini kurarak; CI/CD boru hattı, gözlemlenebilirlik (observability) ve otomatik ölçekleme (autoscaling) entegrasyonlarını hayata geçirmeyi hedefledik.

Takım Motivasyonu ve Katılım Amaçları:

- Şevval Çulcu:** Projenin bulut depolama entegrasyonu, veri yalıtımı, Testcontainers entegrasyon test otomasyonu ve Grafana gözlemlenebilirlik altyapısının kurulması motivasyonu ile projeye katılmıştır.
- Yusuf Durmuş:** Kubernetes ortamında Helm paketleme, KEDA tabanlı metrik odaklı otomatik ölçekleme, ArgoCD GitOps süreçleri ve Playwright ile E2E tarayıcı test otomasyonu mimarisini tasarlama motivasyonu ile çalışmaya dahil olmuştur.

2. Mimari

Sistem mimarimiz, gevşek bağlı (loosely coupled) ve bağımsız olarak ölçeklenebilir bileşenler ilkesine dayanmaktadır. Q-Flow mimarisi, QR kodunun kendisini doğrudan hedef URL'e bağlamak yerine, sistemimizin sunduğu asenkron yönlendirici (`/qr/{id}`) uç noktasına yönlendirir.

Mimari Veri Akışı:

Bileşen Katmanı	Teknoloji	Açıklama
Kullanıcı Arayüzü (Frontend)	HTML / CSS / JS	İstemci tarafında modern glassmorphic tasarımı temel alan responsive arayüz. QR kod yönetimi ve test işlemleri sağlanır.
Backend Servisi	FastAPI / Python	REST API isteklerini yöneten, asenkron uygulama sunucusu. QR kod matrisini üretir, S3 ve PostgreSQL ile iletişim kurar.
İlişkisel Veritabanı	PostgreSQL	QR kodlarına ait metadata bilgilerini (ID, hedef URL, S3 anahtarı, tarama sayısı, tarih) saklayan veritabanı.
Nesne Depolama	LocalStack (AWS S3)	Üretilen PNG formatındaki QR kod görsellerinin bulut depolama standartlarında lokal olarak saklanmasını sağlar.
Orkestrasyon & Altyapı	K8s (Minikube) & Helm	Uygulamanın tek komutta <code>values.yaml</code> parametreleriyle ayağa kalkmasını sağlayan konteyner orkestrasyonu.

Gözlemlenebilirlik	Prometheus / Grafana / Jaeger	Loglama, donanım metrikleri izleme ve OpenTelemetry SDK ile uçtan uca Dağıtık İzleme (Tracing) katmanı.
---------------------------	----------------------------------	---

Kullanıcı (Tarama) → API Gateway (FastAPI) → Metrik Kaydı (PostgreSQL) → Yönlendirme (HTTP 302) → Hedef URL

3. Test Stratejisi

Yazılım kalitesini ve bulut altyapısının doğruluğunu güvence altına almak için test piramidinin tüm katmanları uçtan uca uygulanmıştır. Proje genelinde hedeflenen kod kapsamı (coverage) en az **%70** olarak belirlenmiş, yapılan ölçümlerde **%78** başarı oranına ulaşılmıştır.

Plaintext

```

  ^
 / \   → Uçtan Uca (E2E) & API: Playwright, Postman + Newman (Duman testleri)
/----\
 /   \   → Entegrasyon: Testcontainers (Lokal PostgreSQL & LocalStack S3)
/-----\
/       \ → Birim (Unit): Pytest (Mock S3, In-memory DB, Factory Boy & Faker)
/_____\

```

- **Birim (Unit) Testleri:** Hızlı çalışması ve izole olması amacıyla ilişkisel veritabanı in-memory SQLite ([test.db](#)) ile override edilmiş, AWS S3 çağrıları mock'lanmıştır. [tests/factories.py](#) altında tanımlanan **Factory Boy & Faker** modelleri ile dinamik test verileri üretilmiştir.
- **Entegrasyon Testleri:** Mock'ların ötesine geçerek gerçek sistem bağımlılıklarının doğrulanması için **Testcontainers** kullanılmıştır. Arka planda geçici Docker konteynerleri üzerinde gerçek PostgreSQL ve LocalStack S3 servisleri ayağa kaldırılarak veri/dosya akışları test edilmiştir.
- **Uçtan Uca (E2E) Testler:** **Playwright** ile tarayıcı simülasyonu yapılarak form doldurma, yeni sekmede URL yönlendirme takibi ve DOM üzerinden arayüz güncellemeleri doğrulanmıştır.
- **API Testleri:** CI hattında çalışmak üzere hazırlanan Postman koleksiyonu ile 5'in üzerinde API doğrulama kriterini içeren duman testleri Newman CLI ile koşurulmaktadır.

4. Pipeline & Deploy

Sürekli Entegrasyon (CI) ve Sürekli Dağıtım (CD) süreçleri **GitHub Actions** ile otomatikleştirilmiş olup [.github/workflows/ci.yml](https://github.com/workflows/ci.yml) boru hattı üzerinden çalışmaktadır.

GitHub Actions Akışı:

1. **Linting (Statik Analiz):** Black ve Ruff araçları ile kod standartları denetimi.
2. **Test (Unit & Integration):** Pytest ile testlerin koşulması (Coverage <%70 ise işlem bloke edilir).
3. **Docker Build:** Multi-stage Dockerfile kullanılarak optimize edilmiş [qrcode-app:latest](#) imajının derlenmesi.
4. **Helm Lint & Template:** Dağıtım şablonlarının [helm lint](#) ile sözdizimi doğrulaması.
5. **Newman Smoke Test:** Derlenen imajın ayağa kaldırılarak API sağlık doğrulamasının yapılması.
6. **Deploy:** Tüm adımlar başarılı olduğunda dağıtımın tetiklenmesi.

Kubernetes ve Dağıtım Stratejisi:

- **Helm Chart:** Uygulama ve tüm bağımlılıkları tek bir chart altında paketlenmiş, [values.yaml](#) ile kaynak limitleri ayarlanmış ve *RollingUpdate* stratejisiyle sıfır kesinti (zero-downtime) hedeflenmiştir.
- **KEDA Autoscaling:** Prometheus üzerindeki HTTP istek oranına duyarlı bir [ScaledObject](#) kullanılmış, pod başına saniyede 5 isteği aşan durumlarda replika sayısı 2'den 10'a otomatik ölçeklenmiştir.
- **ArgoCD GitOps:** GitHub deposundaki Helm Chart değişikliklerini dinleyen ArgoCD, küme durumunu Git deposuyla otomatik senkronize ederek *self-healing* ve *pruning* yeteneklerini devreye sokmuştur.

5. Performans & Gözlemlenebilirlik

Mimarinin stres altındaki davranışını ölçmek ve sistem sağlığını izlemek için proaktif metrik yöntemleri kullanılmıştır.

Performans Testi (k6)

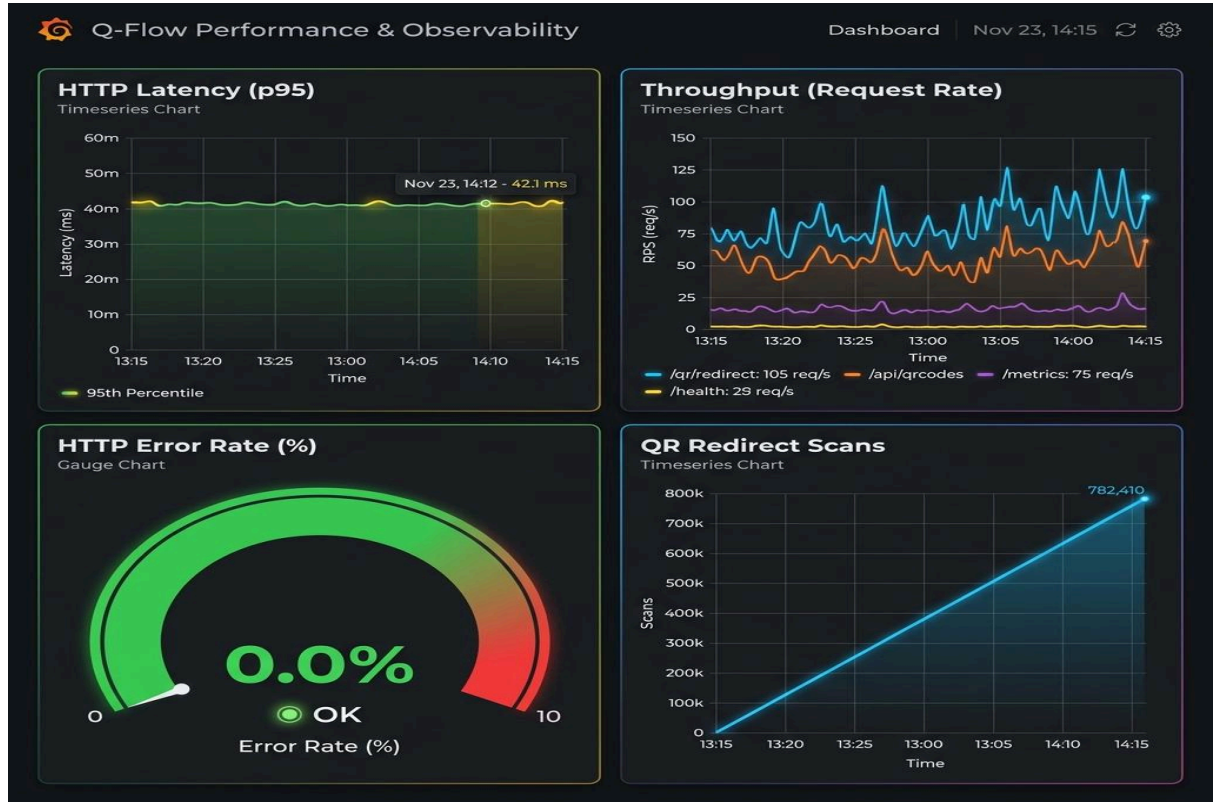
Kubernetes ortamındaki port-forward altyapısı üzerinde koşturulan testte, 10 sanal kullanıcı (VUs) ile uç noktalar yük altına sokulmuştur:

- **Hata Oranı:** %0.00 (Tüm istekler başarıyla HTTP 201 ve HTTP 302 almıştır).
- **p95 Latency:** 625.61 ms (Port-forwarding tünel gecikmesi dahil, < 1000 ms hedefi tutturulmuştur).
- **Toplam İstek:** 484 HTTP isteği başarıyla işlenmiştir.

Gözlemlenebilirlik (Grafana & Jaeger)

Uygulamanın sunduğu **/metrics** uç noktasından alınan verilerle oluşturulan Grafana Dashboard üzerinde; *HTTP Latency (p95)*, *Throughput*, *Error Rate* ve *Active Redirect Scans* anlık olarak izlenmektedir.

Ayrıca **OpenTelemetry** enjeksiyonu sayesinde Jaeger üzerinde, API'den S3 dosya yüklemelerine (boto3) kadar olan mikroservis adımları milisaniye düzeyinde uçtan uca (trace) izlenebilmektedir.



6. Sonuç & Öğrendiklerim

Sayılarla Proje Özeti:

Metrik / İşlem	Gerçekleşen Değer
Kod Kapsamı (Coverage)	%78 (Baraj: %70)
Koşturulan Birim Testi	13 Adet (Pytest)
Koşturulan Entegrasyon Testi	1 Adet (Testcontainers)
Koşturulan E2E Senaryosu	1 Adet (Playwright)
Koşturulan Duman Testi	5 Adet API Kontrolü (Newman)
Performans Testi (k6)	484 İstek (%100 Başarı, %0 Hata Oranı)

Karşılaşılan Zorluklar ve Çözümler:

- Eşzamanlı İsteklerde Unique Constraint Çakışması:** Yük testlerinde S3 için "temp" isimli dosya oluşturmaya çalışan podlar çakışmaya sebep oldu. Çözüm olarak geçici anahtarlar UUID (`temp-uuid`) yapısıyla dinamik hale getirildi.
- Veritabanı Havuzunun Tükenmesi:** SQLAlchemy varsayılan bağlantı limiti aşıldı. `pool_size=25` ve `max_overflow=35` konfigürasyonu ile darboğaz aşıldı.
- Namespace Çözümleme Hatası:** KEDA operatörü, Prometheus servisine erişemediğinde sorun FQDN (Tam nitelikli alan adı: `qrcode-prometheus-service.default.svc.cluster.local:9090`) tanımı yapılarak çözüldü.

İleride Yapılabilecekler:

- LocalStack Pro entegrasyonu ile S3 presigned URL'lerin doğrulanması.
- Kubernetes ortamında log yönetimi için merkezi EFK (Elasticsearch, Fluentd, Kibana) yığınının kurulması.
- Playwright testlerinin CI/CD hattında *headless Chrome* ile çalıştırılması.

7. İş Paylaşımı

Proje, grup üyeleri tarafından **eşit iş yükü (%50 - %50)** paylaşımıyla geliştirilmiş olup görev dağılımı aşağıdaki gibidir:

Ekip Üyesi (Öğrenci No)	Sorumlu Olduğu Modüller ve Geliştirmeler	Commit Oranı
Şevval Çulcu (171423961)	• QR kod üretici motorunun yazılması. • Pytest birim test otomasyonu ve Testcontainers (S3/DB) entegrasyonu. • Dockerfile (multi-stage) ve lokal docker-compose hazırlığı. • Prometheus metrik toplayıcısı ve Grafana dashboard tasarımı. • Final raporu yazımı ve sunum hazırlıkları.	%50

